

Structures

Structures

- ▶ A structure is a collection of one or more variables, possibly of different types, grouped together under a single name for convenient handling.
- ▶ Structures help organize complicated data, particularly in large programs, by treating a group of related variables as a unit instead of separate entities.
- ▶ A structure can be used to represent an ordered pair.

Example: Graphics in C

- ▶ A point can be represented as an ordered pair of coordinates, a rectangle as a pair of points, and so on.
- ▶ The basic object: a point with x and y coordinates (both integers).
- ▶ The structure for a point is declared using the following syntax:

```
struct point {  
    int x;  
    int y;  
};
```

- ▶ The keyword `struct` introduces a structure declaration, which is a list of declarations enclosed in braces.
- ▶ An optional name (like `point`) called structure tag can be used as a shorthand for the declaration.

Structure Members and Tags

- ▶ Variables named in a structure are called members.
- ▶ A structure tag names the kind of structure and can be used as a shorthand for the part of the declaration in braces.
- ▶ A struct declaration defines a type, and it can be followed by a list of variables:

```
struct { ... } x, y, z;
```

- ▶ A structure declaration without variables reserves no storage; it describes a template or shape of a structure.

Alternatives

► Alternative 1:

```
struct {int x; int y;} pt1, pt2;  
// reserves storage for pt1 and pt2
```

► Alternative 2:

```
struct point {int x; int y;};  
// doesn't reserve storage; just a declaration  
  
struct point pt1, pt2;  
// definition of pt1, pt2; reserves storage for them
```

► Alternative 3:

```
typedef struct point {int x; int y;} Point;  
// doesn't reserve storage; just a declaration  
  
Point pt1, pt2;  
// definition of pt1, pt2; reserves storage for them
```

Initializing Structures

- ▶ Structures can be initialized by following their definition with a list of initializers.
- ▶ Example:

```
struct point maxpt = { 320, 200 };
```

- ▶ Automatic structures can be initialized by assignment or by calling a function that returns a structure of the right type.

Accessing Structure Members

- ▶ A member of a structure is referred to in an expression by the form: `structure-name.member`.
- ▶ Example:

```
printf("%d,%d", pt.x, pt.y);
```

Example

```
// Define a structure for a student record
struct Student {
    char name[50];
    int rollNumber;
    float marks;
};

// Declare and initialize a student record
struct Student stu1;
strcpy(stu1.name, "Alice");
stu1.rollNumber = 101;
stu1.marks = 85.5;
```


Example

```
// Another way
typedef struct Student {
    char name[50];
    int rollNumber;
    float marks;
} Stu;

// Now Stu is a type like char or int

// Declare and initialize a student record
Stu stu1;
strcpy(stu1.name, "Alice");
stu1.rollNumber = 101;
stu1.marks = 85.5;
```

Nested Structures

- ▶ Structures can be nested.
- ▶ One representation of a rectangle is a pair of points denoting diagonally opposite corners.
- ▶ Example:

```
struct rect {  
    struct point pt1;  
    struct point pt2;  
};
```

- ▶ If we declare
`struct rect screen;`
then `screen.pt1.x` refers to the x coordinate of the pt1 member of screen.

Example: Payroll Record

- ▶ An employee is described by a set of attributes such as name, address, social security number, salary, etc.
- ▶ Some attributes could be structures themselves; for example, a name can have several components, as can an address and even a salary.

Structure for a Payroll Employee record

```
struct PayrollEmployee {  
    int employeeID;  
    struct name {  
        char firstName[50];  
        char lastName[50];  
    } employeeName;  
    float hourlyRate;  
    float hoursWorked;  
    float totalSalary;  
};
```

Structure for Date

```
int main() {  
    // Example usage of the PayrollEmployee structure  
    struct PayrollEmployee employee1;  
    // Assign values to the members of the structure  
    employee1.employeeID = 1001;  
    sprintf(employee1.employeeName.firstName, "John");  
    sprintf(employee1.employeeName.lastName, "Doe");  
    employee1.hourlyRate = 15.50;  
    employee1.hoursWorked = 40.0;  
    employee1.totalSalary = employee1.hourlyRate  
                            * employee1.hoursWorked;  
    // Print out the details of the employee  
    printf("Employee ID: %d\n", employee1.employeeID);  
    printf("Name: %s %s\n", employee1.employeeName.firstName,  
          employee1.employeeName.lastName);  
    printf("Hourly Rate: $%.2f\n", employee1.hourlyRate);  
    printf("Hours Worked: %.2f\n", employee1.hoursWorked);  
    printf("Total Salary: $%.2f\n", employee1.totalSalary);  
  
    return 0;  
}
```

Structure for a Payroll Employee record

```
// Enum for months
enum Month {
    JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC
};

// Define a structure for a date
struct Date {
    int day;
    enum Month month;
    int year;
};

int main() {
    // Declare and initialize a date
    struct Date today = {22, MAR, 2024};

    // Print the date
    printf("Today's date: %d-%02d-%d\n", today.day,
                                                today.month, today.year);

    return 0;
}
```

Legal Operations on Structures

- ▶ The only legal operations on a structure include copying, assigning, taking its address with `&`, and accessing its members.
- ▶ Copying and assignment, including passing arguments to functions and returning values from functions, are allowed.
- ▶ Structures may not be compared directly.
- ▶ Initialization of structures can be done by a list of constant member values, and automatic structures can be initialized by assignment.

Manipulating Points and Rectangles

- ▶ Let us investigate structures by writing functions to manipulate points and rectangles.
- ▶ Three possible approaches: pass components separately, pass the entire structure, or pass a pointer to it.

Function: makepoint

```
struct point makepoint(int x, int y)
{
    struct point temp;
    temp.x = x;
    temp.y = y;
    return temp;
}
```

- ▶ `makepoint` creates a point structure from `x` and `y` components. It receives components separately, but returns an entire structure.
- ▶ Demonstrates the re-use of names to stress the relationship.

Using makepoint and Example

```
struct rect screen;  
struct point middle;  
struct point makepoint(int, int);  
  
screen.pt1 = makepoint(0, 0);  
screen.pt2 = makepoint(XMAX, YMAX);  
middle = makepoint((screen.pt1.x + screen.pt2.x)/2,  
                  (screen.pt1.y + screen.pt2.y)/2);
```

- ▶ makepoint can be used to initialize structures dynamically or provide structure arguments to a function.

Arithmetic on Points: addpoints

```
struct point addpoint(struct point p1, struct point p2)
{
    p1.x += p2.x;
    p1.y += p2.y;
    return p1;
}
```

- ▶ addpoints adds two points.
- ▶ Both arguments and the return value are structures
- ▶ Note that the structure parameters are passed by value.

Function: ptinrect

```
int ptinrect(struct point p, struct rect r)
{
    return p.x >= r.pt1.x && p.y >= r.pt1.y
        && p.x < r.pt2.x && p.y < r.pt2.y;
}
```

- ▶ ptinrect tests whether a point is inside a rectangle.
- ▶ Assumes the rectangle is presented in canonical form: by specifying the bottom-left and top-right corners.

Function: canonrect

```
struct rect canonrect(struct rect r)
{
    struct rect temp;
    temp.pt1.x = min(r.pt1.x, r.pt2.x);
    temp.pt1.y = min(r.pt1.y, r.pt2.y);
    temp.pt2.x = max(r.pt1.x, r.pt2.x);
    temp.pt2.y = max(r.pt1.y, r.pt2.y);
    return temp;
}
```

- ▶ canonrect canonicalizes coordinates of a rectangle.
- ▶ Uses macros `min` and `max` for minimum and maximum values.

Pointers to Structures

- ▶ If a large structure is to be passed to a function, it is generally more efficient to pass a pointer than to copy the whole structure.
- ▶ Structure pointers are similar to pointers to ordinary variables.
- ▶ Example: `struct point *pp;`

Accessing Structure Members via Pointers

```
struct point origin, *pp;  
pp = &origin;  
printf("origin is (%d,%d)\n", (*pp).x, (*pp).y);
```

- ▶ `(*pp).x` accesses the `x` member of the structure pointed to by `pp`.
- ▶ The parentheses are necessary because the precedence of `.` is higher than `*`.

Alternative Notation for Pointers

- ▶ An alternative notation is provided as a shorthand:
`p->member-of-structure.`
- ▶ Example: `printf("origin is (%d,%d)\n", pp->x, pp->y);`
- ▶ Both `.` and `->` associate from left to right.

Precedence Hierarchy

- ▶ Structure operators `.` and `->`, together with `()` for function calls and `[]` for subscripts, have high precedence.
- ▶ Example: `++p->len` increments `len`, not `p`, because the implied parenthesization is `++(p->len)`.